SHELFLINE — Deadline-Driven Dispatch for Perishable Aid

Solution blueprint, pothole audit and build plan for 'The Last-Mile Spoilage Trap'

0. Brutally honest verdict first

- **Nobody can promise a win.** What you can control: a system that visibly does not break when judges break it, and a repo that proves it was engineered, not pasted together.
- **Most teams will ship a map plus a 'Claim' button.** That is a marketplace. The problem statement is not asking for a marketplace; it is asking for a coordination engine where **time, capacity, compatibility and human unreliability are variables**. UI polish is worth maybe 20% here. Failure handling is worth 80%.
- **Your edge is one invariant enforced everywhere:** no portion is ever dispatched, kept reserved, or delivered after its safe-until time. Every feature below exists to defend that sentence.
- **Biggest risk to you:** over-building the optimizer and under-building the failure demo. A simple greedy planner with excellent recovery beats a perfect solver that crashes on 'shelter closes mid-transit'.
- **Second biggest risk:** a fake-looking demo. Judges will inject failures live. If your system needs a restart, you lose.

1. The idea in one paragraph

SHELFLINE treats every batch of food, medicine or kit as a **ticking clock**, not as inventory. From the donor's cooking time and the item's safety window, it computes a hard `deliver_by` deadline, then plans **backwards**: which riders, which shelters, in which splits, with which fallback if anything changes. Every claim is a **short lease** that expires on its own unless the rider proves progress. Every in-transit rider carries a **pre-computed Plan B and Plan C**, so when a shelter closes mid-journey the system reroutes in under a second instead of starting a search. A **Safety Gate** vetoes anything that could finish late, using pessimistic (P90) travel times rather than averages.

2. Design laws (print these on the wall)

1. **The clock is a resource.** Remaining safe time (slack) is the thing we schedule, spend and protect.
2. **A claim is a lease, never ownership.** Leases expire automatically. Nobody can 'park' a batch.
3. **Cargo in motion always has a Plan B.** A rider is never stranded because the fallback list is precomputed and continuously refreshed.
4. **Safety is a veto, not a score.** A late or incompatible delivery is not a bad option, it is an illegal one.
5. **Everything is an event.** State is rebuilt from an append-only log, so the planner can be re-run at any moment and the demo can be replayed.
6. **Capacity is a ledger, not a flag.** Shelter demand = need − received − committed-in-transit, updated atomically.
7. **Never trust the happy path.** Every API call is idempotent; every timer is persisted; every external dependency has a fallback.

3. Domain model

Entity	Key fields	Notes
Batch	donor_id, category, qty_portions, cooked_at, shelf_life_min, safe_until, pickup_window, cold_chain, allergens, diet_tags	Created by donor; `safe_until` computed, never typed by hand
Lot	batch_id, qty, status	A splittable slice of a batch; the unit that gets leased
Lease	lot_id, rider_id, shelter_id, state, expires_at, last_progress_at, fallbacks[]	The anti-ghost mechanism
Rider	location, vehicle_capacity (portions and volume), insulated_box, reliability_score, status	Reliability is Bayesian, per rider

Entity	Key fields	Notes
Shelter	location, need, received, capabilities{water, fuel, fridge, …}, capabilities_verified_at, intake_open, accepts_diet_tags	Capabilities go stale on purpose
Event	type, payload, ts, actor, idempotency_key	Append-only log; single source of truth

4. The math of time

All times use the **server clock only** (never a phone clock).

- `deliver_by = min(cooked_at + shelf_life, donor_declared_safe_until) − serve_buffer`
  - `serve_buffer` (10–15 min) is the time the shelter needs to actually distribute food.
- `slack(now) = deliver_by − now − P90(rider→donor) − handover − P90(donor→shelter) − handover`
- **Feasible** only if `slack ⩾ 0` using **P90** travel times. Averages kill food. If the P90 arrival is later than `deliver_by`, the triple (rider, batch, shelter) does not exist for the planner.
- `latest_pickup = deliver_by − P90(donor→shelter) − handover`
- `lease_ttl = clamp( P90(rider→donor) + grace, min = 3 min, max = latest_pickup − now − redispatch_reserve )`
  - `redispatch_reserve` = time the next-best rider needs to reach the donor. This is what guarantees that when a lease dies there is still time to recover.
- **Safety note:** the commonly cited rule is that cooked food should not stay in the temperature danger zone beyond roughly 2 hours cumulatively. The problem statement uses 75 minutes, so make shelf-life a per-category config, default conservative, and cite the guideline you used (check FSSAI or WHO guidance and put the link in the README).

ETA service: ship a pluggable interface. v1 = haversine × 1.35 circuity factor ÷ speed profile × traffic multiplier, with noise to produce a distribution. v2 = OSRM or OpenRouteService when reachable. v3 = correct using observed leg times from the simulation. P90 is derived from the distribution, not guessed.

5. The Lease Engine (kills ghost reservations)

5.1 Lease states

`OFFERED → ACCEPTED (soft lease) → ARRIVED_DONOR → PICKED_UP (hard commit, OTP scan) → NEAR_SHELTER → DELIVERED (shelter confirms)`

Failure exits: `EXPIRED`, `RELEASED`, `REVOKED`, `REROUTED`, `ABORTED_UNSAFE`.

5.2 Rules

- **Soft lease** (before pickup): expires at `expires_at`. Any rider progress ping can extend it, but only up to `latest_pickup − redispatch_reserve`. Beyond that it dies no matter what.
- **Hard commit:** only after the donor scans the rider's OTP or QR at the gate. Cargo is physically in transit, so the lease can no longer be reclaimed, only rerouted.
- **Alive is not the same as moving.** Heartbeats alone are not enough (a rider stuck in traffic is alive). Track **progress**: distance-to-donor must shrink across consecutive pings. Three non-shrinking pings = `STALLED`.
- **Escalation ladder** (all driven by persisted deadlines, not in-memory timers):
  1. T+0: lease issued, ETA published.
  2. First missed progress window: nudge (push, then SMS, then an automated call in production).
  3. `STALLED` or second miss: **hot standby** rider is pre-alerted but not yet assigned.
  4. Projected ETA breaches `latest_pickup` or lease expires: lease released, standby promoted, original rider notified politely.
- **Race-safe release:** if the ghost rider reappears after expiry, they get a clear 'lease expired, here is a nearby alternative' and the system does not double-assign.
- **Anti-hoarding:** one active soft lease per rider (two only if the planner proves both are feasible back to back). Riders with repeated expiries get shorter TTLs and lower assignment priority via reliability score.
- **Hot-standby racing (for tight batches):** when slack is below `P90(rider→donor) + redispatch_reserve`, offer to two riders. The first OTP scan wins; the loser is auto-released and immediately re-planned onto another lot. Show riders a small compensation or points credit so this is not demoralizing.

5.3 Reaper

A background loop (every 2–5 s) selects leases where `expires_at < now()` and processes them in a transaction. Because deadlines live in the database, a server restart loses nothing.

6. The Planner (splitting, matching, recovery)

6.1 Pipeline

1. **Safety filter:** enumerate (rider, lot, shelter) triples and drop anything failing the P90 feasibility test, the compatibility rules or capacity.
2. **Greedy EDF pass:** sort lots by earliest `deliver_by` (least slack first), then assign the best feasible rider and shelter by score. This is your always-works baseline and must respond in under 200 ms.
3. **Optional CP-SAT improvement** (OR-Tools) with a hard 1.5 s time limit: maximize safe portions delivered; accept only if it beats greedy by a margin (hysteresis).
4. **Commit** as leases, publish ETAs, attach fallbacks.

Score (higher is better): `value = safe_portions × urgency_weight(shelter) − α·detour − β·P(late) − γ·(1 − rider_reliability)`.

Hysteresis: do not reroute a moving rider unless the gain exceeds a threshold, otherwise the system oscillates and riders lose trust.

6.2 Splitting example (the exact scenario in the handout)

Batch of 180 meals, safe for 75 min, serve buffer 10 min, so `deliver_by = T+65`. Rider capacity 40. Demand: A=50, B=35, C=60, D=40 (total 185).

Rider	Load	Drops
R1	40	A:40
R2	40	A:10, B:30
R3	40	C:40
R4	40	C:20, D:20
R5	20	B:5, D:15

Result: 180 delivered, D left 5 short, and every leg passes the P90 test. Multi-drop is only allowed when the second stop also fits within `deliver_by`. The planner must also say **'I cannot place everything'** rather than silently overpromise: surplus becomes an 'open offer' (see 7.3).

6.3 Infeasible at creation (judges will try this)

A batch with a tight window must **not** be accepted silently. On creation, run feasibility immediately and return one of: `ACCEPTED_FULL`, `ACCEPTED_PARTIAL (n portions placeable)`, or `REJECTED_INFEASIBLE` with the reason ('no rider reachable within window') and the closest viable alternative ('reachable if pickup starts by 20:10', 'only Shelter C is reachable'). Honest rejection scores better than a spoiled delivery.

7. Shelter ledger and mid-transit changes

7.1 Ledger

`available(shelter) = need − received_confirmed − Σ committed_in_transit`

- Updates are atomic (`UPDATE ... SET committed = committed + :q WHERE need − received − committed >= :q`). If zero rows update, the claim fails cleanly. This is what prevents two riders being promised the same last 50 meals.
- Walk-in donations ('a local citizen drops food at Shelter A') are posted by the shelter through a one-tap 'received outside system: +n' button, which immediately recomputes availability and triggers re-evaluation of every in-transit lease heading there.
- Default policy: **no overbooking** for cooked food (waste risk). Optional soft overbook of up to 10% for shelters that flag 'can absorb surplus'.

7.2 Demand closure while a rider is en route (the headline judge test)

Event: `DEMAND_CHANGED(shelter, new_need)` or `SHELTER_CLOSED`.

1. Recompute ledger. Find in-transit leases whose quantity now exceeds availability.
2. For each affected lease, read its **precomputed fallback list** (refreshed every 30 s and on every relevant event), re-validate with current GPS and P90 ETA, and pick the best.
3. If the new destination can only take part of the load, **split in flight**: drop part at the first stop, continue with the rest to the next. The rider app must support partial drops.

4. Push the new route to the rider with a one-tap accept. If there is no acknowledgment in 60 s, escalate (SMS, then call). Auto-accept is allowed when the rider has opted in.
5. Log the reroute with time-to-recover (target: under 1 s to decide, under 60 s to acknowledge).

7.3 The fallback tiers (never leave cargo without an answer)

Tier	Destination	Condition
1	Registered shelter with availability	Passes safety gate and compatibility
2	Overflow sinks: community kitchens, hostel messes, mosques/gurudwaras/temples with langar, registered walk-in points	Pre-registered, open now
3	Open flash offer broadcast to all eligible NGOs and shelters in radius	10-minute soft lease, only if slack allows
4	Safe discard with documented reason	Only when tiers 1–3 are infeasible; counted honestly in waste metrics

Always show the judge which tier fired and why.

8. Compatibility and dependency rules

Put rules in a **declarative config**, not in code branches, so judges see the system is extensible.

```
- item: raw_ration_kit
  requires: [cooking_fuel, clean_water]
- item: infant_formula
  requires: [clean_water, heating_or_kettle]
- item: cooked_meal
  requires: [serving_capacity]
  max_ambient_minutes: 120
- item: cold_chain_medicine
  requires: [refrigeration]
  rider_requires: [insulated_box]
- item: cooked_meal
  diet_must_match: [veg, non_veg, halal, jain]
```

- **Capabilities expire.** Each shelter capability (water, fuel, fridge) has a `verified_at`. After a configurable window (for example 6 h) it becomes `UNKNOWN`, and dependent items are blocked until the shelter re-confirms with one tap. In a flood, 'has clean water' can flip in an hour.
- **Re-validate on every replan,** not only at dispatch. If water runs out while a ration-kit rider is in transit, that triggers the same reroute flow as a closure.
- **Companion dispatch (stretch):** when a shelter needs both kits and water, the planner can create a linked task for water cans instead of just rejecting kits.
- **Vehicle rules:** capacity is checked in portions and volume; cold-chain items require an insulated box; hot food has a shorter ambient limit when not in a hot-box.
- **Medicines:** keep scope narrow (OTC, consumables, supplies) and require a verified pharmacist or consignee at handover. Rules on redistributing medicines are strict in India, so verify the legal position before claiming this feature and state the limitation in your pitch.

9. Failure scenario catalog (your test list)

ID	Situation	Required behavior
S01	Batch created with window shorter than any rider can meet	Immediate infeasibility response with reason and nearest viable option
S02	Batch of 180 vs rider capacity 40 and shelter need 50	Auto-split into lots, multi-rider, multi-drop plan
S03	Rider claims then goes silent	Progress miss, nudge, standby alert, lease expiry, reassign, ghost rider notified
S04	Rider is alive but stuck in traffic	Progress detector flags STALLED; ETA recalculated; standby promoted if P90 breaches
S05	Shelter fills (walk-in donation) while rider is 50% there	Ledger update, instant reroute using precomputed fallback
S06	Shelter closes entirely mid-transit	Tiered fallback; partial drops allowed
S07	Shelter reduces need from 50 to 20 with 40 in transit	Drop 20 there, continue 20 to next best

ID	Situation	Required behavior
S08	Two riders tap Claim in the same millisecond	One wins via atomic update; loser gets instant alternative
S09	Rider phone dies after pickup	Last known position and ETA; shelter alerted; no reclaim (cargo is out); escalate to ops contact
S10	Donor revises quantity down or batch is partly sold or taken	Lots re-balanced; excess leases released; no negative ledger
S11	Batch passes safe-until while unassigned	State EXPIRED; no new leases possible; counted in waste metrics
S12	ETA breaches deliver_by after traffic spike	Rider flagged AT_RISK; reroute to nearer shelter or hot-standby handoff if feasible
S13	Shelter lacks water for raw ration kits	Kits filtered out for that shelter; alternative shelter or companion task
S14	Capability data is stale	Treated as UNKNOWN; one-tap reconfirm request
S15	Rider breaks down mid-route	Rider marks 'breakdown'; nearest standby handoff point; lot reassigned if slack allows
S16	Many events at once (flash flood)	Planner time-boxed; greedy fallback; priority queue by slack
S17	Donor misreports cooking time	Conservative defaults; shelter and rider can reject on arrival with a reason (veto)
S18	Shelter refuses delivery as unsafe	Safety veto recorded; lot marked ABORTED_UNSAFE; no redistribution of that lot
S19	Rider loses connectivity	Offline-tolerant PWA queues events; SMS fallback; server uses last-known state
S20	Server restarts mid-operation	Event log and persisted deadlines restore state; reaper resumes
S21	Same command sent twice (retry)	Idempotency key returns the original result

ID	Situation	Required behavior
S22	Rider tries to deliver after deadline	Gate refuses auto-confirm; shelter decides; flagged as safety exception

10. Pothole audit (traps I expect, with fixes)

#	Pothole	Why it hurts	Fix
1	Using average ETA	Half your deliveries arrive late	Plan on P90 and show the margin on screen
2	Counting only confirmed deliveries in shelter capacity	Over-promises the same space twice	Ledger includes committed in-transit
3	Timers kept in memory	Restart equals ghost reservations return	Deadlines in the DB plus a reaper
4	Heartbeat equals alive equals fine	Stuck riders look healthy	Progress detection (distance shrinking)
5	Race conditions on claims	Judges double-click and break you	Atomic conditional UPDATE or row locks; test it
6	Phone clocks	Skewed timestamps break TTLs	Server time only
7	Mobile browsers throttle background GPS	PWA goes silent when the screen locks	Wake-lock during active jobs, milestone taps (arrived, picked up, near), SMS check-ins
8	Rerouting thrash	Rider gets three instructions in two minutes	Hysteresis and minimum gain threshold
9	Fallback computed only at the moment of failure	Slow and may find nothing	Precompute and refresh continuously

#	Pothole	Why it hurts	Fix
10	Optimizer timeouts under load	Demo freezes during a flood simulation	Anytime planner: greedy first, solver as improvement
11	Trusting donor input on freshness	Unsafe food reaches shelters	Conservative caps, safety veto at pickup and delivery, audit trail
12	Silent acceptance of impossible batches	Judges create one on purpose	Feasibility response at creation
13	Stale shelter capabilities	Kits go to a camp with no water	Capability TTL and re-verification
14	Checking dependencies only at dispatch	Conditions change mid-trip	Re-validate on every event
15	Need expressed in people vs meals	Unit confusion	Store need in portions with a conversion setting
16	Partial deliveries not modeled	Closure scenario has no good answer	Lots, multi-drop, in-flight splits
17	Rider overload	Capacity in count ignores volume and heat	Capacity in portions and volume, insulated-box flag
18	Punishing riders harshly	Volunteers quit	Reliability affects TTL and priority, not shame; thank-you and credit on releases
19	Fake shelters or donors	Malicious or bogus entries	Verified onboarding, OTP handover, shelter confirmation, rate limits
20	Starving distant shelters	Optimizer favors the nearest	Urgency and waiting-time weight in the score
21	Hardcoded demo data	AI evaluator and judges spot it	Seed script plus live chaos console, no scripted outcomes
22	UI-only effort	Looks great, fails tests	Engine tests first, UI second

#	Pothole	Why it hurts	Fix
23	Legal and safety overclaiming (medicines, food safety)	Credibility loss	State assumptions, cite guidance, narrow scope
24	Time-sensitive demo is slow to watch	Judges lose patience	Simulated clock with 10x and 60x speed

11. Architecture and stack

- **Backend:** Python, FastAPI, SQLAlchemy, Postgres (SQLite as a local fallback). Use `SELECT ... FOR UPDATE` or conditional updates for ledgers and leases.
- **Core modules:** `clock`, `events`, `safety_gate`, `ledger`, `leases`, `planner`, `eta`, `rules`, `reaper`, `notifier`.
- **Clock abstraction:** every module calls `clock.now()`. Tests and the demo use a simulated clock you can accelerate or jump.
- **Realtime:** WebSockets for dashboards and rider app updates.
- **Frontend:** React. Three views only: Control Room (map, batches, leases, countdowns), Rider PWA (job card, big buttons, offline queue), Shelter tablet (need, capabilities, one-tap 'received outside' and 'closed').
- **Notifications:** web push plus an SMS gateway mock with a visible 'SMS outbox' panel so judges see the fallback path.
- **Optimization:** OR-Tools CP-SAT as an optional improver behind a flag.
- **Safety Gate:** one function, `is_safe(lot, rider, shelter, now) -> (ok, reasons)`. Every assignment, reroute and confirmation passes through it. Judges can read it in one minute.

12. Chaos Console and judge demo script

Build a visible **Chaos Console** panel (this wins demos because it invites judges to try to break you):

- Ghost this rider
- Traffic jam x3 on this rider
- Close Shelter A now
- Reduce Shelter A need to 20

- Walk-in donation at Shelter A
- Kill rider connection
- Donor reduces quantity
- Double-claim race
- Fast-forward 10 / 20 / 40 min
- Flip shelter water capability to 'no'

Seven-minute script:

1. Create a 180-meal batch with 75 min safe time. Show the auto-split plan and P90 margins. (S02)
2. Create a batch that is impossible. Show instant honest rejection with alternatives. (S01)
3. Ghost a rider. Fast-forward. Show nudge, standby, expiry, reassign, and zero spoiled portions. (S03, S04)
4. Double-claim race live. One wins. (S08)
5. Close Shelter A while a rider is mid-route. Show tier fallback and partial drop. (S05–S07)
6. Flip water off for a shelter with a ration-kit rider en route. Show reroute. (S13, S14)
7. Flood mode: inject 30 batches and 15 riders at once. Planner stays responsive. Show the final scoreboard: portions saved, portions wasted, **spoilage violations = 0**. (S16)

13. Repo, commits and the AI evaluation

The handout says commits are evaluated by an AI. Assume it reads history for genuine progression, tests and clarity.

- **Commit small and often,** roughly every 45–60 minutes of real work, with conventional messages (`feat(lease): add progress-based stall detection`). Do not fake or backdate commits; evaluators detect it and it will cost you more than it gains.
- **Tests named after scenarios:** `test_s06_shelter_closes_mid_transit`, `test_s08_double_claim_race`. This maps your repo directly to the judges' test list.
- **Property-based tests** (Hypothesis) for the core invariants: no delivery completes after `deliver_by`; shelter `committed + received` never exceeds need (unless soft overbook is enabled); one lot never has two active hard leases.
- **CI:** GitHub Actions running tests on every push; add the badge to the README.

- **README must have:** the problem in three lines, architecture diagram (Mermaid), the design laws, a 'break it yourself' section pointing to the Chaos Console, and a one-command setup (`docker compose up` plus `make seed`).
- **docs/DECISIONS.md:** short entries on why leases, why P90, why greedy-first.
- **Honest limitations section:** what is simulated (ETA, SMS) and what would change in production.

14. Build order

Priority	Deliverables	Done when
P0 (must)	Data model, simulated clock, event log, safety gate, shelter ledger, lease engine with reaper, minimal API, core tests	S03, S08, S11, S21 pass automatically
P1	Greedy planner, splitting and multi-drop, precomputed fallbacks, reroute flow, Control Room UI, Chaos Console	S02, S05, S06, S07 demo live
P2	Rider PWA with offline queue, shelter tablet, SMS mock, reliability score, hot-standby racing	S04, S09, S19 pass
P3	CP-SAT improver, rules YAML polish, companion dispatch, fairness weighting, analytics	Only if P0–P2 are rock solid

If time is short, **never trade P0 or P1 for P3.**

15. Metrics to display live

- Portions delivered safely vs portions wasted (waste cause: expired, unsafe, infeasible)
- **Spoilage violations (target: 0)**
- Ghost-lease rate and median time-to-reassign
- Average slack at delivery (how close to the edge you ran)
- Reroute recovery time (decision and acknowledgment)
- Rider utilization and shelter fill percentage

16. What to cut (be ruthless)

- Blockchain or tokens. Not needed and invites questions you cannot answer.
- A trained ML ETA model. A transparent distribution model is easier to defend.
- Complex authentication. Use simple OTP and role switching for the demo.
- Multi-city scale claims. Prove one city's logic first.
- Beautiful map tiles. Clear countdown timers and status colors matter more.

17. Pitch in three sentences

'Food does not fail because people are unwilling; it fails because time, capacity and trust are not modeled. SHELFLINE turns every batch into a countdown, every claim into an expiring lease, and every rider into someone with a pre-computed Plan B. Break it live: we have a console for exactly that, and our invariant is that no portion is ever delivered unsafe.'